MongoDB

for Beginners

A Complete Guide to NoSQL Database Programming

Theory • Syntax • Code • Explanation • Examples

Prepared by

Mohanraj R

Python & SQL Trainer | B.Tech IT

GUVI Certified Python Programmer | HackerRank Python Gold Badge

Chennai

Table of Contents

Table of Contents 2
Chapter 1: Introduction to Databases & NoSQL 4
1.1 What is a Database? 4
1.2 SQL vs NoSQL 4
1.3 Types of NoSQL Databases 4
1.4 What is MongoDB? 5
1.5 Why Use MongoDB? (Key Features) 5
Chapter 2: MongoDB Architecture 6
2.1 Core Concepts: Database → Collection → Document 6
2.2 MongoDB vs SQL Terminology 6
2.3 BSON — The Storage Format 6
2.4 A Sample MongoDB Document 6
Chapter 3: Installation & Getting Started 8
3.1 What You Need 8
3.2 Starting the Shell 8
3.3 Basic Shell Navigation Commands 8
Chapter 4: Databases and Collections 9
4.1 Creating / Switching a Database 9
4.2 Creating a Collection 9
4.3 Dropping a Collection or Database 9
Chapter 5: CRUD — Create (Insert Documents) 11
5.1 insertOne() — Insert a Single Document 11
5.2 insertMany() — Insert Multiple Documents 11
5.3 Understanding _id and ObjectId 12
Chapter 6: CRUD — Read (Query Documents) 13
6.1 find() and findOne() 13
6.2 Comparison Query Operators 13
6.3 Logical Operators 14
6.4 Projection — Selecting Specific Fields 14
6.5 sort(), limit(), skip() 15
Chapter 7: CRUD — Update Documents 16
7.1 updateOne() and updateMany() 16
7.2 Common Update Operators 16
7.3 replaceOne() — Replace an Entire Document 16
7.4 upsert — Update or Insert 17

Chapter 8: CRUD — Delete Documents 18
8.1 deleteOne() and deleteMany() 18
Chapter 9: MongoDB Data Types (BSON Types) 19
Chapter 10: Embedded Documents & Arrays 20
10.1 Embedding vs Referencing 20
10.2 Querying Embedded Fields (Dot Notation) 20
10.3 Querying Arrays 20
Chapter 11: Indexing 22
11.1 Creating an Index 22
11.2 Types of Indexes 22
Chapter 12: Aggregation Framework 23
12.1 Common Pipeline Stages 23
12.2 Example Aggregation Pipeline 23
Chapter 13: Mini Project — Student Management with MongoDB 24
Step 1: Design the Collection 24
Step 2: Practice Queries 24
Chapter 14: Mini Project — Python + MongoDB (Student Management App) 26
14.1 Setup 26
14.2 Connecting to MongoDB from Python 26
14.3 Full Project Code — Student Management Console App 26
14.4 Explanation — How It Works 28
14.5 Sample Run 29
14.6 Extend This Project (Ideas for Students) 29
Chapter 15: Quick Reference Cheat Sheet 30
Chapter 16: Practice Exercises for Students 31

Chapter 1: Introduction to Databases & NoSQL

1.1 What is a Database?

THEORY

A database is an organized collection of data that can be easily accessed, managed, and updated. Every application you use — a college portal, a shopping app, a chat app — stores its data in a database behind the scenes.

So far you have learned SQL databases like MySQL / SQL Server. These are called Relational Databases (RDBMS) because data is stored in tables made of rows and columns, and tables are related to each other using keys.

1.2 SQL vs NoSQL

THEORY

NoSQL stands for "Not Only SQL." It is a different way of storing data — not in rigid tables, but in flexible formats such as documents, key-value pairs, wide columns, or graphs. MongoDB is the most popular NoSQL database, and it stores data as documents (similar to JSON).

Aspect	SQL (RDBMS)	NoSQL (MongoDB)
Data structure	Tables (rows & columns)	Collections of Documents (JSON-like)
Schema	Fixed schema, defined in advance	Flexible / dynamic schema
Relationships	Uses JOINs across tables	Uses embedding or references
Best suited for	Structured, transactional data	Rapidly changing, hierarchical data
Examples	MySQL, SQL Server, Oracle	MongoDB, Cassandra, Redis, Neo4j
Query language	SQL	MQL (MongoDB Query Language) / JS-like syntax

1.3 Types of NoSQL Databases

THEORY

NoSQL databases are grouped into four main types, based on how they store data:

Type	How data is stored	Example databases
Document Store	Data stored as documents (JSON/BSON)	MongoDB, CouchDB
Key-Value Store	Simple key → value pairs	Redis, DynamoDB
Column-Family Store	Data stored in column groups	Cassandra, HBase
Graph Database	Data stored as nodes & relationships	Neo4j, ArangoDB

Our focus: MongoDB → a Document Store database.

1.4 What is MongoDB?

THEORY

MongoDB is an open-source, document-oriented NoSQL database. Instead of storing data in tables, MongoDB stores data as documents — which look very similar to Python dictionaries or JSON objects. This makes it very natural to use with Python, JavaScript, and web applications.

1.5 Why Use MongoDB? (Key Features)

- Flexible Schema — each document in a collection can have different fields.
- Document-Oriented — stores data as BSON (Binary JSON), close to how data looks in code.
- Scalability — designed to scale horizontally across many servers (sharding).
- High Performance — fast reads/writes, good for large, fast-changing data.
- Rich Query Language — supports filtering, sorting, aggregation, indexing.
- Widely used in the industry — Uber, eBay, and many startups use MongoDB.

Chapter 2: MongoDB Architecture

2.1 Core Concepts: Database → Collection → Document

THEORY

MongoDB organizes data in a three-level hierarchy:

DATABASE *e.g. schoolDB — a container for collections*
↓ *contains*
COLLECTION *e.g. students — similar to a SQL Table*
↓ *contains*
DOCUMENT *e.g. { name: "Arun", age: 17 } — similar to a SQL Row*

2.2 MongoDB vs SQL Terminology

Since you already know SQL, this table maps every concept you know to its MongoDB equivalent:

SQL (RDBMS) Term	MongoDB Term
Database	Database
Table	Collection
Row / Record	Document
Column	Field
Primary Key	_id (auto-generated by default)
JOIN	$lookup (in Aggregation) or Embedding
Schema	No fixed schema (flexible)

2.3 BSON — The Storage Format

THEORY

MongoDB does not store data as plain text JSON. It converts and stores it internally as BSON (Binary JSON). BSON supports more data types than plain JSON (like Date and binary data), and it is faster for the database to read and write.

2.4 A Sample MongoDB Document

CODE

```
{
  "_id": ObjectId("64fa1c2e5b3f9a001e8b4567"),
  "name": "Arun Kumar",
  "age": 17,
  "class": "12th",
  "subjects": ["Python", "Maths", "SQL"],
  "address": {
    "city": "Chennai",
    "pincode": 600001
  }
}
```

EXPLANATION

This single document represents one student record. Notice:

- _id — a unique identifier MongoDB auto-creates for every document.
- subjects — an array (list), stored directly inside the document.
- address — an embedded document (a document inside a document, similar to a nested dictionary).

This is exactly why MongoDB feels natural for Python learners — a document looks just like a Python dictionary with nested lists and dicts.

Chapter 3: Installation & Getting Started

3.1 What You Need

Tool	Purpose
MongoDB Community Server	The actual database engine (runs in the background)
mongosh (Mongo Shell)	Command-line tool to run MongoDB commands
MongoDB Compass	Free GUI tool to view & manage data visually (like SSMS for SQL Server)

Download all three free from mongodb.com. After installing, MongoDB runs as a background service on your machine (default port: 27017).

3.2 Starting the Shell

SYNTAX

```
mongosh
```

EXPLANATION

Typing mongosh in your terminal connects you to the local MongoDB server and opens an interactive shell where you can type MongoDB commands directly — similar to opening the SQL Server / MySQL command line.

3.3 Basic Shell Navigation Commands

Command	What it does
show dbs	Lists all databases on the server
use schoolDB	Switches to (or creates) a database called schoolDB
show collections	Lists all collections in the current database
db	Shows the name of the current database
db.students.help()	Shows all methods available on the students collection

Chapter 4: Databases and Collections

4.1 Creating / Switching a Database

THEORY

Unlike SQL Server (CREATE DATABASE ...), MongoDB does not need an explicit create command. You simply "use" a database name, and MongoDB creates it automatically the first time you insert data into it.

SYNTAX

```
use <database_name>
```

CODE

```
use schoolDB
```

Output:

```
switched to db schoolDB
```

EXPLANATION

This switches the active database to schoolDB. If schoolDB does not exist yet, MongoDB will remember the name but will only actually create it once you insert the first document.

4.2 Creating a Collection

THEORY

A collection is like a SQL table but with no fixed structure. You can create one explicitly, or let MongoDB create it automatically the first time you insert a document.

SYNTAX

```
db.createCollection("collection_name")
```

CODE

```
db.createCollection("students")
```

Output:

```
{ ok: 1 }
```

4.3 Dropping a Collection or Database

SYNTAX & CODE

```
db.students.drop()        // deletes the 'students' collection
db.dropDatabase()         // deletes the current database
```

EXPLANATION

Use these carefully — both operations permanently delete data. Good to demonstrate on a throwaway/test database during class.

Chapter 5: CRUD — Create (Insert Documents)

CRUD stands for Create, Read, Update, Delete — the four basic operations every database supports. We will cover each one in detail, exactly like we did with SQL's INSERT, SELECT, UPDATE, DELETE.

5.1 insertOne() — Insert a Single Document

THEORY

insertOne() adds exactly one document to a collection. If the collection does not exist yet, MongoDB creates it automatically.

SYNTAX

```
db.collection_name.insertOne({ field1: value1, field2: value2, ... })
```

CODE

```
db.students.insertOne({
  name: "Divya",
  age: 16,
  class: "11th",
  city: "Chennai"
})
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId("64fa1d3f5b3f9a001e8b4570")
}
```

EXPLANATION

acknowledged: true confirms the write succeeded. insertedId shows the auto-generated _id MongoDB assigned to this new document, since we did not provide one ourselves.

5.2 insertMany() — Insert Multiple Documents

SYNTAX

```
db.collection_name.insertMany([ {doc1}, {doc2}, {doc3} ])
```

CODE

```
db.students.insertMany([
  { name: "Arun", age: 17, class: "12th", city: "Chennai" },
  { name: "Priya", age: 15, class: "10th", city: "Madurai" },
  { name: "Karthik", age: 16, class: "11th", city: "Coimbatore" }
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    "0": ObjectId("64fa1e.."), "1": ObjectId("64fa1f.."), "2": ObjectId("64fa20..")
  }
}
```

EXPLANATION

insertMany() takes an array of documents and inserts all of them in one call — much faster than calling insertOne() repeatedly.

5.3 Understanding _id and ObjectId

THEORY

Every MongoDB document must have a unique _id field — this is the equivalent of a SQL Primary Key. If you don't provide one, MongoDB auto-generates a 12-byte ObjectId that is guaranteed to be unique.

CODE

```
db.students.insertOne({ _id: "S101", name: "Kavya", age: 17 })  // custom _id
```

EXPLANATION

You can also assign your own _id (like a roll number), as long as it stays unique within the collection — inserting a duplicate _id will throw a duplicate key error.

Chapter 6: CRUD — Read (Query Documents)

6.1 find() and findOne()

THEORY

find() returns all matching documents (like SELECT * in SQL). findOne() returns only the first matching document.

SYNTAX

```
db.collection_name.find( <query>, <projection> )
db.collection_name.findOne( <query> )
```

CODE

```
db.students.find()                        // all documents
db.students.find({ class: "11th" })       // filtered, like WHERE class="11th"
db.students.findOne({ name: "Arun" })     // only the first match
```

Output:

```
{ _id: ObjectId("..."), name: "Karthik", age: 16, class: "11th", city: "Coimbatore" }
```

EXPLANATION

The object passed to find() is the filter/query condition — directly equivalent to the SQL WHERE clause.

6.2 Comparison Query Operators

THEORY

To filter using conditions like greater-than or not-equal, MongoDB uses special keys starting with $, called operators.

Operator	Meaning	SQL Equivalent
$eq	Equal to	=
$ne	Not equal to	!=
$gt	Greater than	>
$gte	Greater than or equal	>=
$lt	Less than	<
$lte	Less than or equal	<=
$in	Matches any value in a list	IN
$nin	Matches none of the values in a list	NOT IN

CODE

```
db.students.find({ age: { $gt: 15 } })
db.students.find({ class: { $in: ["10th", "11th"] } })
```

EXPLANATION

The first query finds all students older than 15. The second finds students who are in either 10th or 11th class — same idea as SQL's WHERE class IN ('10th','11th').

6.3 Logical Operators

Operator	Meaning
$and	All conditions must be true
$or	At least one condition must be true
$not	Negates a condition
$nor	None of the conditions should be true

CODE

```
db.students.find({
  $or: [ { class: "12th" }, { age: { $lt: 15 } } ]
})
```

EXPLANATION

This returns students who are either in 12th class OR younger than 15.

6.4 Projection — Selecting Specific Fields

THEORY

Just like SQL's SELECT name, age instead of SELECT *, MongoDB uses a second argument to find() called projection to choose which fields to return.

SYNTAX

```
db.collection.find( <query>, { field1: 1, field2: 1, _id: 0 } )
```

CODE

```
db.students.find({}, { name: 1, class: 1, _id: 0 })
```

Output:

```
{ name: "Arun", class: "12th" }
{ name: "Divya", class: "11th" }
```

EXPLANATION

1 means “include this field”, 0 means “exclude this field.” Here we show only name and class, and hide the default _id.

6.5 sort(), limit(), skip()

SYNTAX & CODE

```
db.students.find().sort({ age: 1 })     // 1 = ascending, -1 = descending
db.students.find().limit(3)             // only first 3 documents
db.students.find().skip(2)              // skip first 2 documents
db.students.find().sort({ age: -1 }).limit(5)   // chaining methods
```

EXPLANATION

These methods can be chained together, similar to ORDER BY, TOP/LIMIT, and OFFSET in SQL.

Chapter 7: CRUD — Update Documents

7.1 updateOne() and updateMany()

THEORY

Update operations modify existing documents. You must use update operators (like $set) — you cannot simply overwrite fields directly.

SYNTAX

```
db.collection.updateOne( <filter>, { $set: { field: newValue } } )
db.collection.updateMany( <filter>, { $set: { field: newValue } } )
```

CODE

```
db.students.updateOne(
  { name: "Arun" },
  { $set: { class: "12th - A Section" } }
)
```

Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

EXPLANATION

The first argument is the filter (which document to update), the second is the update operation. $set changes only the specified field(s) and leaves everything else untouched.

7.2 Common Update Operators

Operator	Purpose	Example
$set	Set/update a field's value	{ $set: { age: 18 } }
$unset	Remove a field	{ $unset: { city: "" } }
$inc	Increase/decrease a numeric field	{ $inc: { age: 1 } }
$rename	Rename a field	{ $rename: { "class": "grade" } }
$push	Add a value into an array field	{ $push: { subjects: "AI" } }
$pull	Remove a value from an array field	{ $pull: { subjects: "AI" } }

7.3 replaceOne() — Replace an Entire Document

CODE

```
db.students.replaceOne(
  { name: "Priya" },
```

```
  { name: "Priya", age: 16, class: "10th", city: "Trichy" }
)
```

EXPLANATION

Unlike updateOne(), replaceOne() replaces the whole document (except _id) with the new one you provide.

7.4 upsert — Update or Insert

THEORY

Setting upsert: true tells MongoDB: “if no document matches the filter, insert a new one instead of doing nothing.”

CODE

```
db.students.updateOne(
  { name: "Meena" },
  { $set: { age: 17, class: "12th" } },
  { upsert: true }
)
```

Chapter 8: CRUD — Delete Documents

8.1 deleteOne() and deleteMany()

THEORY

Delete operations remove documents matching a filter permanently. Always double-check the filter before running a delete!

SYNTAX

```
db.collection.deleteOne( <filter> )
db.collection.deleteMany( <filter> )
```

CODE

```
db.students.deleteOne({ name: "Meena" })
db.students.deleteMany({ class: "10th" })
db.students.deleteMany({})     // deletes ALL documents, keeps the collection
```

Output:

```
{ acknowledged: true, deletedCount: 1 }
```

EXPLANATION

deleteOne() removes only the first match; deleteMany() removes every matching document. Passing an empty filter {} to deleteMany() clears the whole collection, but the collection itself still exists (unlike drop()).

Chapter 9: MongoDB Data Types (BSON Types)

THEORY

Every field in a document has a data type, just like columns in SQL have types like VARCHAR or INT. Below are the most common BSON types you will use:

Type	Example	Notes
String	"Chennai"	Text data, written in double quotes
Integer (Int32 / Int64)	17	Whole numbers
Double	89.5	Decimal numbers
Boolean	true / false	Logical values
Array	["Python","SQL"]	Ordered list of values
Object / Embedded Document	{ city: "Chennai" }	A document nested inside another
Null	null	Represents “no value”
Date	ISODate("2026-07-18")	Stores date & time
ObjectId	ObjectId("64fa...")	Default type for _id, auto-generated
Binary Data	BinData(...)	Used for images, files, etc.

CODE

```
db.students.insertOne({
  name: "Sanjay",
  age: 16,
  percentage: 91.5,
  isActive: true,
  subjects: ["Python", "C++", "SQL"],
  joinedOn: new Date(),
  guardian: { name: "Mr. Ravi", phone: "9876543210" }
})
```

EXPLANATION

This single document uses six different BSON types together — string, number, double, boolean, array, date, and an embedded document — showing how flexible a MongoDB document can be.

Chapter 10: Embedded Documents & Arrays

10.1 Embedding vs Referencing

THEORY

In SQL, related data is split across tables and joined. In MongoDB, you have two design choices:

Approach	How it works	Best for
Embedding	Store related data inside the same document	Data that always belongs together (e.g. student + address)
Referencing	Store an _id reference to a document in another collection	Data that is large, shared, or changes independently (e.g. student + school)

10.2 Querying Embedded Fields (Dot Notation)

SYNTAX

```
db.collection.find({ "parentField.childField": value })
```

CODE

```
db.students.find({ "address.city": "Chennai" })
```

EXPLANATION

Dot notation lets you reach inside a nested/embedded document to filter on its inner fields, just like accessing student["address"]["city"] in a Python dictionary.

10.3 Querying Arrays

Operator	Meaning
$all	Array must contain all specified values
$size	Array must have an exact number of elements
$elemMatch	At least one array element matches all given conditions

CODE

```
db.students.find({ subjects: "Python" })              // array contains "Python"
db.students.find({ subjects: { $all: ["Python", "SQL"] } })
db.students.find({ subjects: { $size: 3 } })
```

EXPLANATION

MongoDB automatically searches inside arrays, so subjects: "Python" matches any document where "Python" is one of the values in that array — no special syntax needed for a simple containment check.

Chapter 11: Indexing

THEORY

An index is a special data structure that speeds up searches, exactly like an index at the back of a textbook. Without an index, MongoDB has to scan every document in a collection (a “collection scan”) to find matches — slow for large collections.

11.1 Creating an Index

SYNTAX

```
db.collection.createIndex({ field: 1 })   // 1 = ascending, -1 = descending
```

CODE

```
db.students.createIndex({ name: 1 })
```

Output:

```
name_1
```

EXPLANATION

This creates an ascending index on the name field. Future queries filtering or sorting by name will now be much faster.

11.2 Types of Indexes

Index Type	Description
Single Field Index	Index on one field — db.students.createIndex({ age: 1 })
Compound Index	Index on multiple fields together — db.students.createIndex({ class: 1, age: -1 })
Unique Index	Prevents duplicate values — db.students.createIndex({ email: 1 }, { unique: true })
Text Index	Enables full-text search on string fields — db.students.createIndex({ name: "text" })

EXPLANATION

The _id field always has a unique index automatically — that's how MongoDB enforces that every _id is unique without you doing anything.

Chapter 12: Aggregation Framework

THEORY

Aggregation is MongoDB's way of doing GROUP BY, SUM, AVG, and multi-step data processing — similar to SQL's GROUP BY combined with aggregate functions. Data flows through a pipeline of stages, one after another, like a factory assembly line.

12.1 Common Pipeline Stages

Stage	Purpose	SQL Equivalent
$match	Filters documents	WHERE
$group	Groups documents & computes totals/averages	GROUP BY
$sort	Sorts documents	ORDER BY
$project	Reshapes documents / selects fields	SELECT column list
$limit	Limits number of results	TOP / LIMIT

12.2 Example Aggregation Pipeline

SYNTAX

```
db.collection.aggregate([ { stage1 }, { stage2 }, ... ])
```

CODE

```
db.students.aggregate([
  { $match: { class: { $in: ["11th", "12th"] } } },
  { $group: { _id: "$class", totalStudents: { $sum: 1 }, avgAge: { $avg: "$age" } } },
  { $sort: { totalStudents: -1 } }
])
```

Output:

```
{ _id: "12th", totalStudents: 4, avgAge: 16.75 }
{ _id: "11th", totalStudents: 3, avgAge: 16.0 }
```

EXPLANATION

Step by step:

- $match keeps only students in 11th or 12th class.
- $group groups the remaining documents by class, counting students ($sum: 1) and averaging their age ($avg: "$age").
- $sort arranges the grouped results by totalStudents, highest first.

This single pipeline replaces what would otherwise need several separate SQL queries.

Chapter 13: Mini Project — Student Management with MongoDB

THEORY

Let's apply everything learned so far to build a small, realistic collection design and a set of operations — exactly the kind of exercise to do live in class.

Step 1: Design the Collection

CODE

```
use schoolManagement

db.students.insertMany([
  { name: "Arun", age: 17, class: "12th", city: "Chennai",
    subjects: ["Python", "SQL", "Maths"], percentage: 88.5 },
  { name: "Divya", age: 16, class: "11th", city: "Madurai",
    subjects: ["Python", "English"], percentage: 91.2 },
  { name: "Karthik", age: 17, class: "12th", city: "Chennai",
    subjects: ["SQL", "Physics"], percentage: 76.0 },
  { name: "Priya", age: 15, class: "10th", city: "Trichy",
    subjects: ["Maths", "Science"], percentage: 95.4 }
])
```

Step 2: Practice Queries

CODE

```
// 1. All 12th class students
db.students.find({ class: "12th" })

// 2. Students scoring above 85%, sorted highest first
db.students.find({ percentage: { $gt: 85 } }).sort({ percentage: -1 })

// 3. Update Karthik's percentage after re-evaluation
db.students.updateOne({ name: "Karthik" }, { $set: { percentage: 79.5 } })

// 4. Add a new subject to Divya
db.students.updateOne({ name: "Divya" }, { $push: { subjects: "AI Basics" } })

// 5. Average percentage per class
db.students.aggregate([
  { $group: { _id: "$class", avgPercentage: { $avg: "$percentage" } } }
])
```

EXPLANATION

Ask students to predict the output of each query before running it — this builds intuition for how filters, updates, and aggregation stages behave.

Chapter 14: Mini Project — Python + MongoDB (Student Management App)

THEORY

MongoDB works beautifully with Python because a document looks exactly like a Python dictionary. We connect to MongoDB from Python using a library called pymongo. Below is a complete, runnable console application that performs full CRUD on a students collection.

14.1 Setup

CODE

```
pip install pymongo
```

EXPLANATION

pymongo is the official Python driver for MongoDB. Make sure your MongoDB server is running locally (default: mongodb://localhost:27017) before running the script below.

14.2 Connecting to MongoDB from Python

SYNTAX

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["database_name"]
collection = db["collection_name"]
```

EXPLANATION

MongoClient() opens a connection to the server. Indexing into it with a database name (like a dictionary key) gives you that database, and indexing into the database gives you a collection — no separate “connect to database” step is needed, and both are created automatically on first insert.

14.3 Full Project Code — Student Management Console App

CODE

```
from pymongo import MongoClient

# ---------- Connect to MongoDB ----------
client = MongoClient("mongodb://localhost:27017/")
db = client["schoolManagement"]
students = db["students"]


def add_student():
```

```
    name = input("Enter name: ")
    age = int(input("Enter age: "))
    student_class = input("Enter class: ")
    city = input("Enter city: ")
    subjects = input("Enter subjects (comma separated): ").split(",")

    student = {
        "name": name,
        "age": age,
        "class": student_class,
        "city": city,
        "subjects": [s.strip() for s in subjects]
    }
    result = students.insert_one(student)
    print(f"Student added with id: {result.inserted_id}")



def view_all_students():
    all_students = students.find()
    print("\n--- All Students ---")
    for s in all_students:
        print(f"{s['_id']} | {s['name']} | Age: {s['age']} | Class: {s['class']} |
City: {s['city']} | Subjects: {s['subjects']}")



def search_student():
    name = input("Enter name to search: ")
    student = students.find_one({"name": name})
    if student:
        print(f"\nFound: {student}")
    else:
        print("No student found with that name.")



def update_student():
    name = input("Enter name of student to update: ")
    new_percentage = float(input("Enter new percentage: "))
    result = students.update_one(
        {"name": name},
        {"$set": {"percentage": new_percentage}}
    )
    if result.matched_count:
        print("Student updated successfully.")
    else:
        print("No matching student found.")
```

```
def delete_student():
    name = input("Enter name of student to delete: ")
    result = students.delete_one({"name": name})
    if result.deleted_count:
        print("Student deleted successfully.")
    else:
        print("No matching student found.")


def menu():
    while True:
        print("\n===== Student Management (MongoDB + Python) =====")
        print("1. Add Student")
        print("2. View All Students")
        print("3. Search Student by Name")
        print("4. Update Student Percentage")
        print("5. Delete Student")
        print("6. Exit")
        choice = input("Enter your choice (1-6): ")

        if choice == "1":
            add_student()
        elif choice == "2":
            view_all_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            update_student()
        elif choice == "5":
            delete_student()
        elif choice == "6":
            print("Goodbye!")
            break
        else:
            print("Invalid choice, try again.")


if __name__ == "__main__":
    menu()
```

14.4 Explanation — How It Works

- MongoClient(...) connects once at the top of the script; students refers to the collection we work with throughout.
- add_student() builds a Python dictionary from user input and inserts it with insert_one() — exactly the same insert_one() you ran in mongosh, just called from Python.

- view_all_students() loops over the cursor returned by find() — in pymongo, find() returns an iterable cursor, not a list, so we read it with a for loop.
- search_student() uses find_one() with a filter dictionary, identical in structure to the filters used in Chapter 6.
- update_student() uses update_one() with the $set operator, same as Chapter 7 — pymongo simply lets you build that same query as a Python dict.
- delete_student() uses delete_one(), and checks deleted_count to confirm whether a document was actually removed.

The key teaching point: every MongoDB shell command from earlier chapters maps directly onto a pymongo method with the same name and the same filter/update dictionary shape — students who understand the mongosh syntax already know 90% of pymongo.

14.5 Sample Run

Output:

```
===== Student Management (MongoDB + Python) =====
1. Add Student
2. View All Students
3. Search Student by Name
4. Update Student Percentage
5. Delete Student
6. Exit
Enter your choice (1-6): 1
Enter name: Sanjay
Enter age: 16
Enter class: 11th
Enter city: Chennai
Enter subjects (comma separated): Python, SQL
Student added with id: 64fb2a1e5b3f9a001e8b45aa
```

14.6 Extend This Project (Ideas for Students)

- Add input validation (e.g. reject empty names, negative ages).
- Add a filter option: view only students from a chosen class or city.
- Add sorting: view students sorted by percentage using .sort().
- Connect this to a simple Tkinter GUI instead of a console menu.
- Use aggregate() to add a “class topper report” option to the menu.

Chapter 15: Quick Reference Cheat Sheet

Operation	Command
Show all databases	show dbs
Switch/create database	use dbName
Show collections	show collections
Create collection	db.createCollection("name")
Insert one document	db.col.insertOne({...})
Insert many documents	db.col.insertMany([{...},{...}])
Find all documents	db.col.find()
Find with filter	db.col.find({field: value})
Find one document	db.col.findOne({field: value})
Update one document	db.col.updateOne({filter},{$set:{...}})
Update many documents	db.col.updateMany({filter},{$set:{...}})
Delete one document	db.col.deleteOne({filter})
Delete many documents	db.col.deleteMany({filter})
Sort results	db.col.find().sort({field: 1})
Limit results	db.col.find().limit(n)
Create an index	db.col.createIndex({field: 1})
Run aggregation	db.col.aggregate([{stage1},{stage2}])
Drop collection	db.col.drop()
Drop database	db.dropDatabase()

Chapter 16: Practice Exercises for Students

Give these as in-class or homework exercises using the schoolManagement.students collection from Chapter 13:

- 1. Insert 3 more student documents, including at least one field the others don't have (e.g. "scholarship: true").
- 2. Find all students from "Chennai" who are older than 16.
- 3. Find all students who study both "Python" and "SQL".
- 4. Increase every student's percentage by 2 marks using $inc.
- 5. Remove the "city" field from one student using $unset.
- 6. Write an aggregation pipeline that finds the topper (highest percentage) in each class.
- 7. Create a unique index on a "rollNumber" field, then try inserting two students with the same rollNumber and observe the error.
- 8. Delete all students below 15 years of age.
- 9. Extend the Python project in Chapter 14 with a new menu option “View Toppers” that prints every student scoring above 90%.

End of Guide — Happy Teaching!